# LAPORAN PRAKTIKUM
# PEMROGRAMAN BERORIENTASI OBJEK
# MODUL 7 – ABSTRACT dan INTERFACE




NAMA : Ruhfaidah Shirotul Akhiroh

NIM : 103132400019

KELAS : S1SD-05-REG-01

**PROGRAM STUDI SAINS DATA**

**FAKULTAS INFORMATIKA**

**TELKOM UNIVERSITY PURWOKERTO**

**2026**

A. Dasar Teori

1. Kelas Abstract

   Kata kunci abstrak digunakan untuk *method* atau *class* yang belum memiliki implementasi. Abstrak *method* dideklarasikan pada abstrak class. Kelas yang dideklarasikan sebagai abstrak tidak akan bisa dibentuk *Object* dalam Java. Sehingga kelas abstrak tidak akan bisa digunakan di dalam program yang lain, kecuali kelas abstrak tersebut diturunkan dan turunan dari kelas abstrak tersebut yang dapat dijadikan *Object*.

2. Interface

   *Interface* serupa dengan kelas. Kelas mendefinisikan kemampuan sebuah *Object*, sementara *interface* mendefinisikan *method* atau konstanta yang akan diimplementasikan pada *Object* yang lain. *Interface* membantu mendefinisikan sifat *Object* dengan mendeklarasikan seperangkat karakteristik *Object* tersebut.

   *Interface* pada Java 8 memiliki beberapa batasan:

   a. Semua atribut adalah *public*, *static* dan *final* (semua atribut bertindak sebagai konstanta).
   b. Hanya memiliki *default method*, dan *static method*.
   c. Tidak boleh ada deklarasi konstruktor.

   Berbeda dengan *class* biasa, *interface* mengizinkan *multiple inheritance*, yaitu sebuah kelas dapat mempunyai dua *interface* induk sekaligus. Hal tersebut tidak dapat dilakukan pada sebuah kelas, dimana sebuah kelas hanya boleh mempunyai satu kelas induk. *Syntax* yang diperlukan hampir sama dengan cara membuat kelas. Perbedaannya hanya *method* pada *interface* tidak memiliki isi/ variabel.

   *Interface* sebelum Java 8 hanya dapat memiliki *method* abstrak. Semua *method* di *Interface* bersifat publik dan abstrak secara *default*. Namun pada Java 8 memungkinkan *interface* untuk memiliki *method default* dan *static*. Hal tersebut dikarenakan *default method* pada *interface* dapat memungkinkan pengembang menambahkan *method* baru ke *interface* tanpa mempengaruhi kelas yang mengimplementasikan *interface*.

3. Perbedaan Abstract Class dan Interface

   a. Semua interface Method tidak memiliki body sedangkan beberapa abstract class dapat memiliki Method dengan implementasi.
   b. Sebuah *interface* hanya dapat mendefinisikan *constant* sedangkan sebuah *abstract class* tampak seperti kelas biasa yang dapat mendeklarasikan variabel.
   c. *Interface* tidak memiliki hubungan *inheritance* secara langsung dengan sebuah *class* tertentu, sedangkan *abstract class* bisa jadi hasil turunan dari *abstract class* induknya.

d. *Interface* memungkinkan terjadinya pewarisan jamak (*multiple inheritance*) sedangkan *abstract class* tidak.

B. Guided

1. Abstract

```
package com.week7.guided.gd1;

public abstract class Employee {
    private String name;

    public Employee(String name){
        this.name = name;
    }

    public String getName(){
        return name;
    }

    // Setiap pegawai memiliki gaji, namun rumus berbeda-beda
    public abstract double earnings();
}
```

```
package com.week7.guided.gd1;

public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary){
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings(){
        return weeklySalary;
    }
}
```

```
package com.week7.guided.gd1;

public class MainGd1 {
    public static void main(String[] args) {
        SalariedEmployee salaried = new SalariedEmployee("Budi", 5000000);
        System.out.println("Nama: "+salaried.getName());
        System.out.println("Gaji: Rp"+salaried.earnings());
```

```
    }
}
```

Output :

```
Nama: Budi
Gaji: Rp5000000.0
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  2.363 s
Finished at: 2026-10-30T13:58:29+07:00
------------------------------------------------------------------------
```

2. Interface

```
package com.week7.guided.gd2;

public abstract class Employee2 implements Payable{
    private String name;

    public Employee2(String name){
        this.name = name;
    }

    public String getName(){
        return name;
    }

    // Setiap pegawai memiliki gaji, namun rumus berbeda-beda
    public abstract double earnings();

    // Saat ada penagihan, alihkan langsung ke motode kalkulasi gaji (earnings)
    @Override
    public double getPaymentAmount(){
        return earnings();
    }
}
```

```
package com.week7.guided.gd2;

public interface Payable {
    double getPaymentAmount();

}
```

```
package com.week7.guided.gd2;
```

```
public class SalariedEmployee extends Employee2{
   private double weeklySalary;

   public SalariedEmployee(String name, double weeklySalary) {
      super(name);
      this.weeklySalary = weeklySalary;
   }

   @Override
   public double earnings() {
      return weeklySalary;
   }
}
```

```
package com.week7.guided.gd2;

public class Invoice implements Payable {
   private String partName;
   private int quantity;
   private double pricePerItem;

   public Invoice(String partName, int quantity, double pricePerItem){
      this.partName = partName;
      this.quantity = quantity;
      this.pricePerItem = pricePerItem;
   }

   public String getPartName(){
      return partName;
   }

   // Hitung tagihan
   @Override
   public double getPaymentAmount(){
      return quantity * pricePerItem;
   }
}
```

```
package com.week7.guided.gd2;

public class Maingd2 {
   public static void main(String[] args) {
      Payable[] items = new Payable[2];
      items[0] = new Invoice("Monitor LED", 2, 1500000);
      items[1] = new SalariedEmployee("Budi", 5000000);

      // Dynamic dispatch: Java otomatis mengeksekusi rumus yang tepat sesuai objek
      for (Payable item : items){
         System.out.println("jumlah yang perlu dibayar: Rp"+item.getPaymentAmount());
```

```
    }
  }
}
```

Output :

```
jumlah yang perlu dibayar: Rp3000000.0
jumlah yang perlu dibayar: Rp5000000.0
------------------------------------------------
BUILD SUCCESS
------------------------------------------------
Total time:  1.156 s
Finished at: 2026-10-30T14:01:11+07:00
------------------------------------------------
```

3. [judul guided 3]

```
package com.week7.guided.gd3;

public interface Payable3 {
   double getPaymentAmount();

   // Fitur varu
   default void printReceipt(){
      System.out.println("Struk Resmi - Total: Rp"+getPaymentAmount());
   }

   static void logTransaction(String message){
      System.out.println("[AUDIT LOG] "+message);
   }
}
```

```
package com.week7.guided.gd3;

public abstract class Employee2 implements Payable3{
   private String name;

   public Employee2(String name) {
      this.name = name;
   }

   public String getName() {
      return name;
   }

   // Setiap pegawai memiliki gaji, namun rumus berbeda-beda
   public abstract double earnings();

   // Saat ada penagihan, alihkan langsung ke motode kalkulasi gaji (earnings)
```

```
    @Override
    public double getPaymentAmount() {
        return earnings();
    }
}
```

```
package com.week7.guided.gd3;

public class SalariedEmployee extends Employee2 {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }
}
```

```
package com.week7.guided.gd3;

public class Invoice implements Payable3 {
    private String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double pricePerItem){
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName(){
        return partName;
    }

    // Hitung tagihan
    @Override
    public double getPaymentAmount(){
        return quantity * pricePerItem;
    }
}
```

```
package com.week7.guided.gd3;
```

```
public class Maingd3 {
    public static void main(String[] args) {
        Payable3.logTransaction("Sistem pembayaran diinisialisasi");

        Payable3[] items = new Payable3[2];
        items[0] = new Invoice("Ram DDP 5 16GB", 2, 2000000);
        items[1] = new SalariedEmployee("Budi", 5000000);

        for (Payable3 item : items) {
            item.printReceipt();
        }

        Payable3.logTransaction("Transaksi selesai diproses");
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ modul-07-ab
[AUDIT LOG] Sistem pembayaran diinisialisasi
Struk Resmi - Total: Rp4000000.0
Struk Resmi - Total: Rp5000000.0
[AUDIT LOG] Transaksi selesai diproses
------------------------------------------------
BUILD SUCCESS
------------------------------------------------
Total time:  1.140 s
Finished at: 2026-10-30T14:03:37+07:00
------------------------------------------------
```

C. Unguided

1. Soal Unguided 1

   Sebuah bioskop memerlukan sistem kasir yang mampu memproses pembayaran dalam satu keranjang belanja. Di kasir, penonton tidak hanya membeli Tiket Nonton, tetapi juga bisa membeli makanan/minuman (Paket Snack F&B).

   Secara konsep sistem, Tiket Bioskop dan Paket Snack berasal dari silsilah keluarga (hierarki) yang sepenuhnya berbeda. Tiket film murni pun bersifat abstrak (tidak bisa dibeli tanpa jenis studio). Namun, di mata program kasir, tiket dan makanan harus bisa disatukan, dihitung total tagihannya, dan dicetak di dalam satu struk secara otomatis.

   Pengerjaan soal ini dibagi menjadi 4 tugas utama untuk melatih abstraksi kelas, antarmuka (interface), dan fitur modern Java 8+.

   Jawaban :

```
package com.week7.unguided;

public interface ItemTransaksi {
    // Kontrak publik kasir
    double getNominalBayar();
    String getNamaTransaksi();

    // Mencetak satu baris struk
    default void cetakRincianItem(){
        System.out.println("- "+getNamaTransaksi()+" | Total: Rp"+getNominalBayar());
    }

    // Mencetak log kasir
    static void catatLogKasir(String pesan){
        System.out.println("[LOG SISTEM KASIR] "+pesan);
    }
}
```

```
package com.week7.unguided;

public abstract class TiketBioskop implements ItemTransaksi {
    private String judulFilm;
    private double hargaDasar;
    private int jumlahKursi;

    public TiketBioskop(String judulFilm, double hargaDasar, int jumlahKursi){
        this.judulFilm = judulFilm;
        this.hargaDasar = hargaDasar;
        this.jumlahKursi = jumlahKursi;
    }

    public String getJudulFilm(){
        return judulFilm;
    }

    public double getHargaDasar(){
        return hargaDasar;
    }

    public int getJumlahKursi(){
        return jumlahKursi;
    }

    // Diisi oleh class turunan
    public abstract double hitungTotalBayar();

    @Override
    public abstract String getNamaTransaksi();

    @Override
```

```
    public double getNominalBayar(){
        return hitungTotalBayar();
    }
}
```

```
package com.week7.unguided;

public class TiketReguler extends TiketBioskop {
    private int nomorStudio;

    public TiketReguler(String judulFilm, double hargaDasar, int jumlahKursi, int
nomorStudio){
        super(judulFilm, hargaDasar, jumlahKursi);
        this.nomorStudio = nomorStudio;
    }

    @Override
    public double hitungTotalBayar(){
        return getHargaDasar() * getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi(){
        return "Tiket reguler (Studio " + nomorStudio + ") - " + getJudulFilm() +
                " (" + getJumlahKursi() + " x Rp" + getHargaDasar() + ")";
    }
}
```

```
package com.week7.unguided;

public class TiketVIP extends TiketBioskop {
    private double biayaFasilitasVIP;

    public TiketVIP(String judulFilm, double hargaDasar, int jumlahKursi, double
biayaFasilitasVIP){
        super(judulFilm, hargaDasar, jumlahKursi);
        this.biayaFasilitasVIP = biayaFasilitasVIP;
    }

    @Override
    public double hitungTotalBayar(){
        return (getHargaDasar() + biayaFasilitasVIP) * getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi(){
        return "Tiket VIP Premiere - " + getJudulFilm() +
                " (" + getJumlahKursi() + " x Rp" + (getHargaDasar() + biayaFasilitasVIP) +
")";
```

```
    }
}
```

```
package com.week7.unguided;

public class PaketSnack implements ItemTransaksi {
    private String namaPaket;
    private int jumlahPorsi;
    private double hargaPerPorsi;

    public PaketSnack(String namaPaket, int jumlahPorsi, double hargaPerPorsi){
        this.namaPaket = namaPaket;
        this.jumlahPorsi = jumlahPorsi;
        this.hargaPerPorsi = hargaPerPorsi;
    }

    @Override
    public double getNominalBayar(){
        return jumlahPorsi * hargaPerPorsi;
    }

    @Override
    public String getNamaTransaksi() {
        return "F&B Snack: " + namaPaket + " (" + jumlahPorsi + " x Rp" + hargaPerPorsi +
")";
    }
}
```

```
package com.week7.unguided;

public class Main {
    public static void main(String[] args){
        // 1. Uji roteksi abstraksi: kelas abstract tidak bisa di-instansiasi
        // TiketBioskop tiket = new TiketBioskop("Interstellar", 40000, 2);

        // 2. Buka kasir (static method dipanggil lewat nama interface)
        ItemTransaksi.catatLogKasir("Membuka transaksi baru untuk Pelanggan #042.");

        // 3. Buat objek transaksi
        TiketReguler tiketReguler = new TiketReguler("Interstellar", 40000, 2, 3);
        TiketVIP tiketVIP = new TiketVIP("Dune: Part Two", 60000, 2, 25000);
        PaketSnack snack = new PaketSnack("Combo Popcorn Caramel + 2 Soda", 1,
55000);

        // 4. Array polimorfik (heterogen)
        ItemTransaksi[] keranjang = new ItemTransaksi[3];
        keranjang[0] = tiketReguler;
        keranjang[1] = tiketVIP;
        keranjang[2] = snack;
```

```
        // 5. Pemrosesan dinamis
        System.out.println("=== DAFTAR TRANSAKSI BIOSKOP ===");
        double grandTotal = 0;
        for (ItemTransaksi item : keranjang){
            item.cetakRincianItem();
            grandTotal += item.getNominalBayar();
        }

        // 6. Tutup kasir
        System.out.println("------------------------------------------------------------");
        System.out.println("TOTAL TRANSAKSI : Rp" + grandTotal);

System.out.println("============================================================
============");
        ItemTransaksi.catatLogKasir("Transaksi #042 selesai diproses.");
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ modul-07-abstrac-dan-interfac ---
[LOG SISTEM KASIR] Membuka transaksi baru untuk Pelanggan #042.
=== DAFTAR TRANSAKSI BIOSKOP ===
- Tiket reguler (Studio 3) - Interstellar (2 x Rp40000.0) | Total: Rp80000.0
- Tiket VIP Premiere - Dune: Part Two (2 x Rp85000.0) | Total: Rp170000.0
- F&B Snack: Combo Popcorn Caramel + 2 Soda (1 x Rp55000.0) | Total: Rp55000.0
------------------------------------------------------------
TOTAL TRANSAKSI : Rp305000.0
============================================================
[LOG SISTEM KASIR] Transaksi #042 selesai diproses.
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  2.203 s
Finished at: 2026-10-30T22:11:17+07:00
------------------------------------------------------------------------
```

Penjelasan :

1. ItemTransaksi.java (Interface)

   *Interface* ini adalah standar yang ahrus dipenuhi semua barang di kasir.

   - *double getNominalBayar();* dan *String getNamaTransaksi();* adalah *method* abstrak. *Interface* hanya mendeklarasikannya tanpa isi, dan kelas yang mengimplementasikannya wajib mengisinya.
   - *default void cetakRincianItem()* adalah *default method* (Java 8). *Method* ini punya isi, jadi semua kelas yang mengimplementasikan *interface* otomatis memilikinya tanpa perlu menulis ulang. Di dalamnya ia memanggil *getNamaTransaksi()* dan *getNominalBayar(),* dan versi yang dijalankan menyesuaikan objek yang memanggilnya.

- *static void catatLogKasir(String pesan)* adalah *static method* (Java 8). *Method* ini dipanggil lewat *interface* (*ItemTransaksi.catatLogKasir(...)*), bukan lewat objek.

2. TiketBioskop.java (Abstract Class)
   - *abstract* berarti kelas ini tidak bisa di-instansiasi (*new TiketBioskop(...)* akan error). sehingga hanya menjadi cetakan untuk kelas turunan.
   - Tiga *field* (*judulFilm, hargaDasar, jumlahKursi*) bersifat *private* (enkapsulasi) dan hanya bisa diakses lewat getter.
   - Di konstruktor, parameter diberi nama sama dengan *field*, sehingga perlu *this.judulFilm = judulFilm*; agar Java tahu mana *field* dan mana parameter.
   - *hitungTotalBayar()* dan *getNamaTransaksi()* adalah *abstract method*. Rumusnya berbeda untuk tiap jenis tiket, jadi pengisiannya diserahkan ke kelas turunan.
   - *getNominalBayar()* adalah *method* dari *interface* yang diisi di sini dengan return *hitungTotalBayar();*. Dengan begitu *hitungTotalBayar()* tetap menjadi logika internal tiket, sedangkan *getNominalBayar()* menjadi pintu publik untuk kasir.
   - *@Override* menandai bahwa *method* tersebut menimpa *method* milik *interface* atau induk.

3. TiketReguler.java
   - Extends berarti kelas ini turunan dari *TiketBioskop*.
   - *Field* tambahannya adalah *nomorStudio*.
   - *super(judulFilm, hargaDasar, jumlahKursi);* memanggil konstruktor induk untuk mengisi tiga *field* milik induk. *Field nomorStudio* diisi sendiri.
   - Karena *field* induk *private*, *hitungTotalBayar()* memakai getter: *getHargaDasar() * getJumlahKursi().*
   - *getNamaTransaksi()* menyusun teks nama transaksi sesuai format soal.

4. TiketVIP.java
   Strukturnya mirip dengan kelas *TiketReguler*, bedanya ada *field biayaFasilitasVIP* (biaya sofa recliner per kursi).
   - *hitungTotalBayar()* adalah *(hargaDasar + biayaFasilitasVIP) * jumlahKursi*. Biaya VIP dijumlahkan dulu ke harga per kursi, baru dikali jumlah kursi.
   - Pada *getNamaTransaksi()*, harga satuan yang tampil sudah termasuk biaya VIP (60000 + 25000 = 85000).

5. PaketSnack.java

- Kelas ini bukan turunan *TiketBioskop* , kelas ini berdisi sendiri dan hanya mengimplementasikan *interface ItemTransaksi*.
- Karena langsung mengimplementasikan *interface*, sehingga wajib mengisi *getNominalBayar() (jumlahPorsi * hargaPerPorsi)* dan *getNamaTransaksi().*
- *Method cetakRincianItem()* tidak perlu ditulis ulang karena sudah diwarisi dari *default method interface*.

6. Main.java
   a. Uji proteksi abstraksi. Baris *new TiketBioskop(...)* dijadikan komentar. Kalau komentarnya dibuka, *code* akan error karena kelas *abstract* tidak bisa dibuat objeknya.
   b. Buka kasir. *ItemTransaksi.catatLogKasir(...)* dipanggil lewat nama *interface* karena *static method*.
   c. Buat tiga objek: satu *TiketReguler*, satu *TiketVIP*, dan satu *PaketSnack*.
   d. Array polimorfik. *ItemTransaksi[] keranjang = new ItemTransaksi[3];* bisa menampung ketiga objek walaupun jenisnya berbeda, karena semuanya mengimplementasikan *ItemTransaksi*.
   e. Loop. *for (ItemTransaksi item : keranjang)* menelusuri tiap elemen array;
      - *Item.cetakRincianItem()* mencetak satu baris struk.
      - *grandTotal += item.getNominalBayar()* menjumlahkan total belanja.
   f. Tutup kasir. Program mencetak total (80000 + 170000 + 55000 = 305000), lalu memanggil *catatLogKasir* lagi.

D. Kesimpulan

*Abstract class* digunakan sebagai kerangka umum untuk tiket. Kelas ini tidak bisa dibuat objeknya, dan rumus perhitungan diserahkan ke kelas turunan melalui *abstract method*. *Interface* berfungsi sebagai kontrak yang menyatukan kelas dari hierarki berbeda, yaitu tiket dan snack, sehingga keduanya bida diproses bersama di kasir. Polimorfisme terlihat saat array bertipe *ItemTransaksi[]* menampung objek berbeda jenis, kemudian satu perulangan memanggil *cetakRincianItem()* dan *getNominalBayar()* pada masing-masing objek dengan hasil sesuai jenisnya.

Fitue Java 8 berupa *default method* (*cetakRincianItem()*) dan *static method* (*catatLogKasir()*) memungkinkan *interface* diberi fungsi tambahan tanpa mengubah kelas yang sudah mengimplementasikannya. Perbedaan *abstract class* dan *interface* yaitu *abstract class* cocok untuk kelas yang punya hubungan pewarisan dan berbagi atribut, sedangkan *interface* cocok untuk menyatukan kelas yang tidak berhubungan lewat satu kontrak.